

Universidad Tecnológica Nacional (UTN) Tecnicatura Universitaria en Programación

Materia: Programación 2

Proyecto: Trabajo Práctico Integrador - Food Store (Sistema de Gestión de Pedidos de Comida)

Equipo de Desarrollo:

- Insaurrealde, Hugo Adrian
- Ortiz, Lucia
- Montero, Juan Nahuel

Fecha de Entrega: 22 de Junio de 2026

Índice

1. Marco Teórico	1
2. Modelo UML	1
3. Decisiones Técnicas y Arquitectura	2
4. Dificultades y Soluciones	4
5. Conclusiones	4
6. Bibliografía / Webgrafía y Links	4

1. Marco Teórico

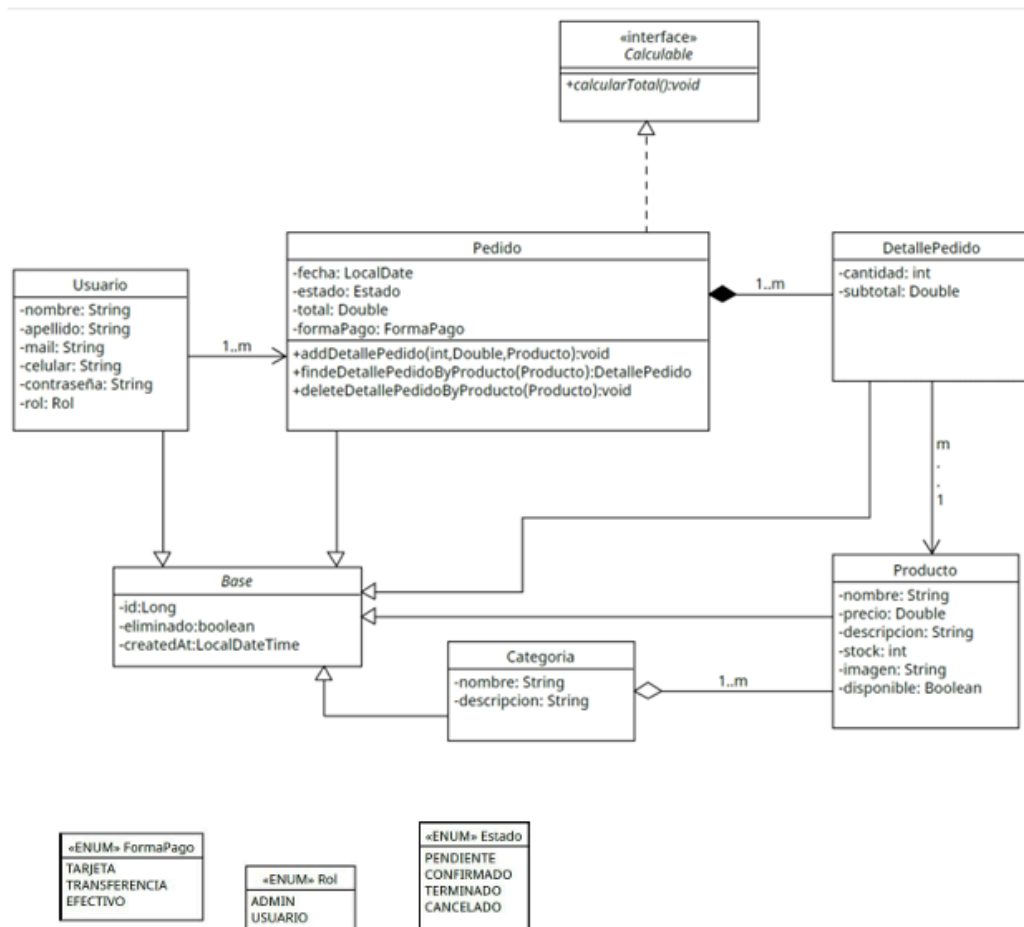
Para el desarrollo del sistema **Food Store**, se utilizó el lenguaje de programación **Java** (versión 21), implementando los pilares fundamentales de la Programación Orientada a Objetos (POO):

- **Herencia y Polimorfismo:** Se definió una clase abstracta `Base` de la cual heredan todas las entidades del sistema, estandarizando atributos como identificadores y estados lógicos.
- **Encapsulamiento:** El acceso a los atributos de las clases está restringido, garantizando que la modificación del estado de los objetos se realice exclusivamente a través de sus respectivos métodos y validaciones.
- **Estructuras de Datos Dinámicas:** Dado que el sistema no cuenta con una base de datos persistente, la información se almacena en memoria durante el tiempo de ejecución haciendo uso del framework de **Colecciones de Java** (específicamente la interfaz `List` implementada a través de `ArrayList`), lo que permite un manejo dinámico del tamaño de los datos.

2. Modelo UML

El diseño del dominio respeta la base del diagrama de clases provisto por la cátedra, pero fue extendido por el equipo para incorporar una relación 1 a 1 unidireccional efectiva entre las entidades Pedido y Factura. De esta manera, el modelo abarca de

forma completa asociaciones 1..N, N..1 y 1..1, vinculando de manera cohesiva a los Usuarios, Pedidos, Detalles de Pedido, Productos, Categorías y Facturas.



3. Decisiones Técnicas y Arquitectura

Para mantener el código ordenado y evaluable, se adoptó una arquitectura basada en capas, logrando un alto nivel de cohesión y un bajo acoplamiento. El proyecto fue estructurado en los siguientes paquetes principales:

- **Entities:** Contiene el modelo de dominio. Se implementó una clase abstracta Base que centraliza los atributos comunes (id, eliminado, createdAt). Entidades como Usuario, Producto, Categoria, Pedido y Factura heredan de esta clase, garantizando la reutilización de código.
- **Service:** Actúa como la capa de lógica de negocio. Clases como UsuarioService y PedidoService encapsulan las estructuras de datos en memoria y centralizan las validaciones (por ejemplo, evitar correos duplicados o stock negativo). Esto evita que la clase principal quede sobrecargada.
- **Exceptions:** Se desarrollaron excepciones personalizadas, tales como EntidadNoEncontradaException y ValidacionException, para un manejo robusto de errores. En lugar de detener la ejecución ante un dato inválido, los servicios lanzan estas excepciones que son capturadas en la interfaz de consola.
- **Enums e Interfaces:** Se utilizaron enumeraciones (Rol, Estado, FormaPago) para restringir de forma segura las variables críticas. Se implementó la interfaz

Calculable en la clase `Pedido`, definiendo el contrato del método `CalcularTotal()`.

Decisiones Clave de Diseño (POO):

1. **Delegación de Responsabilidades:** El cálculo matemático no se realiza de forma procedimental en los servicios. El objeto `DetallePedido` calcula su propio subtotal instanciándose y solicitando el precio al objeto `Producto`.
2. **Baja Lógica (Soft Delete):** No se utilizan métodos `.remove()` en las colecciones. La eliminación se gestiona a través del atributo booleano `eliminado`, y los métodos de listado filtran los registros inactivos para no mostrarlos.
3. **Relación 1 a 1 Unidireccional Efectiva:** Se implementó una asociación directa entre `Pedido` y `Factura` sin el uso de estructuras dinámicas (colecciones), garantizando que a cada pedido le corresponda un único comprobante fiscal. Para asegurar la fluidez de la interfaz de consola y no sobrecargar al operador, la factura se instancia automáticamente con valores por defecto a través del ciclo de vida del pedido en memoria.

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: a
Error: Por favor, ingrese un numero valido.

=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: |
```

```

--- GESTION DE USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al Menu Principal
Seleccione: 2
Ingrese nombre: Hugo
Ingrese apellido: Insaurrealde
Ingrese email: Hugo_123@gmail.com
Ingrese celular: 1122334455
Ingrese contrasenia: 1234
Ingrese Rol (ej. ADMIN, USUARIO): USUARIO
❖exito! Usuario creado con el ID: 1

```

```

--- GESTION DE USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al Menu Principal
Seleccione: 1
--- LISTADO DE USUARIOS ---
ID: 1 | Nombre: Hugo Insaurrealde | Mail: Hugo_123@gmail.com | Rol: USUARIO

```

4. Dificultades y Soluciones

Durante el ciclo de desarrollo, el equipo se enfrentó a diversos desafíos técnicos que fueron resueltos aplicando buenas prácticas de programación:

- Validación de Entradas en Consola:** Al utilizar la clase `Scanner` para la navegación del menú, el ingreso accidental de caracteres de texto en opciones numéricas generaba una interrupción abrupta del sistema (`InputMismatchException`). La solución consistió en envolver la lectura de datos en bloques `try-catch`, capturando específicamente el `NumberFormatException` (utilizando `Integer.parseInt(scanner.nextLine())`) y mostrando un mensaje de error amigable, retornando al usuario al menú principal sin que el programa colapse.
- Gestión de Excepciones Personalizadas:** Inicialmente, las validaciones de reglas de negocio (como evitar duplicidad de correos o stock insuficiente) se manejaban con impresiones de consola simples. Para mejorar la robustez, se aislaron estas reglas en la capa de servicios creando la excepción `ValidacionException`, logrando separar la lógica del sistema de la interfaz de usuario.
- Adaptación Dinámica del Modelo de Dominio (Relación 1-1):** Durante la revisión de los criterios rigurosos de evaluación, el equipo detectó la necesidad de evidenciar una relación 1 a 1 real en el código, una asociación que no se encontraba explícita en el diagrama de clases del enunciado simplificado.

- **Solución:** Se analizó el negocio y se decidió diseñar e incorporar la entidad Factura. El desafío técnico fue integrarla sin alterar la lógica de negocio ya consolidada en PedidoService. Se resolvió mediante una estrategia de acoplamiento seguro, haciendo que Pedido autoinstancie su Factura en segundo plano, cumpliendo con la pauta estricta de evaluación de forma transparente para el usuario de la consola.

-

5. Conclusiones

El desarrollo del proyecto *Food Store* permitió consolidar de manera práctica los fundamentos de la Programación Orientada a Objetos en Java. La implementación de una arquitectura en capas facilitó la distribución de tareas dentro del equipo, asegurando que la lógica de negocio, el modelo de datos y la vista (consola) funcionaran de manera independiente pero integrada. Asimismo, el manejo de colecciones y el diseño de excepciones a medida resultaron fundamentales para construir una aplicación tolerante a fallos, robusta y escalable.

6. Bibliografía / Webgrafía y Links

- **Documentación Oficial de Oracle:** Java 21 API Specification.
- **Material de Estudio UTN:** Módulos y apuntes oficiales de la cátedra de Programación 2.

Enlaces del Proyecto:

- **Video Demostración:**
- **Repositorio publico Git Hub:**
https://github.com/HugoInsaurrealde/Tpi_Programacion2/tree/main